

Day 18: Templates & Zero-Cost Abstractions

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Function Templates

A **function template** generates a family of functions — one for each type it's called with. The compiler generates the specific version at compile time:

```
template <typename T>
T max_val(T a, T b) { return (a > b) ? a : b; }

max_val<uint8_t>(3, 7);    // Generates: uint8_t max_val(uint8_t, uint8_t)
max_val<float>(1.5f, 2.3f); // Generates: float max_val(float, float)
```

No runtime overhead — the compiler generates specific code for each type, just like hand-written functions.

Class Templates

A **class template** generates a family of classes parameterized by type and/or size:

```
template <typename T, size_t N>
class StaticQueue { /* ... */ };

StaticQueue<uint8_t, 16>  uart_buf;    // 16-byte queue
StaticQueue<uint32_t, 8>  adc_buf;     // 8x 32-bit queue
```

Both are different types, compiled independently, with no runtime polymorphism overhead.

C RTP — Curiously Recurring Template Pattern

C RTP achieves **static polymorphism** — polymorphic behavior resolved at compile time, with no vtable:

```
template <typename Derived>
class SensorBase {
public:
    bool init() { return static_cast<Derived*>(this)->do_init(); }
    bool read(int32_t *v) { return static_cast<Derived*>(this)->do_read(v); }
};

class TempSensor : public SensorBase<TempSensor> {
public:
    bool do_init() { /* hardware init */ return true; }
    bool do_read(int32_t *v) { *v = 2350; return true; }
};
```

`SensorBase::init()` calls `TempSensor::do_init()` — resolved at compile time. Zero vtable overhead.

Template

Board-specific behavior can be a template parameter — you select the board at compile time, not runtime:

```
template <typename PinConfig>
class SpiDriver {
    static constexpr uint8_t CS_PIN = PinConfig::CS_PIN;
    static constexpr uint8_t CLK_PIN = PinConfig::CLK_PIN;
};

struct nRF52840PinConfig { static constexpr uint8_t CS_PIN = 10; static constexpr uint8_t CLK_PIN = 11; };
SpiDriver<nRF52840PinConfig> spi;
```

static_assert with Templates

`static_assert` inside templates catches wrong template arguments at compile time:

```
template <typename T, size_t N>
class CircularBuffer {
    static_assert(N > 0, "Buffer size must be > 0");
    static_assert((N & (N-1)) == 0, "Buffer size must be power of 2");
    // ...
};
```

```
CircularBuffer<uint8_t, 7> bad_buf; // COMPILE ERROR: not power of 2
CircularBuffer<uint8_t, 8> good_buf; // OK
```

Zero-Overhead Abstractions

C++ templates produce **zero-overhead abstractions** — the abstraction layer adds no runtime cost compared to writing it by hand. This is why embedded C++ can be as efficient as C when used correctly.

Practice Tasks

1. Create a `template class MovingAverage` filter — instantiate for `uint16_t` (ADC) and `float` (temperature)
 2. Use CRTP to implement a `Logger` base class that `UartLogger` and `RttLogger` derive from
 3. Add `static_assert` constraints to your `CircularBuffer` — test that bad arguments produce clear compile errors
 4. Benchmark CRTP virtual call vs real virtual call using DWT cycle counter — verify zero overhead
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day18
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day18/main.cpp](#)